

29.3 Cross-validation

The criteria of [Section 29.2](#) correct the training error by a formula that rests on assumptions. Cross-validation instead predicts cases that the fit has not seen. It needs neither σ^2 nor degrees of freedom, and applies to any procedure, including one that selects a model. Its price is computation, a bias that is easy to describe, and a variance that is not.

29.3.1 K-fold and leave-one-out

Definition 29.3.1. Cross-validation

Let the cases $(\mathbf{x}_{(i)}, y_i)$, $i = 1, \dots, n$, be divided into K disjoint **folds** $I_1, \dots, I_K$. For each k , let $\hat{f}^{(-k)}$ be the prediction rule obtained by applying the procedure to the cases outside I_k . The **K -fold cross-validation** estimate of prediction error is

$$CV_K = \frac{1}{n} \sum_{k=1}^K \sum_{i \in I_k} (y_i - \hat{f}^{(-k)}(\mathbf{x}_{(i)}))^2.$$

With $K = n$, each fold is a single case, and CV_n is **leave-one-out** cross-validation. With $K = 2$ and one fold used only for fitting, the estimate is a **holdout** or **validation-set** estimate.

The idea is old (Stone 1974; Geisser 1975; Allen 1974 for least squares). What cross-validation estimates is clearest when the cases are a random sample.

Proposition 29.3.1. What K-fold cross-validation estimates

Let the cases be independent draws of $(\mathbf{X}, Y)$, let the folds be fixed in advance with equal sizes n/K , and let $\text{Err}(m) = E(Y_0 - \hat{f}_m(\mathbf{X}_0))^2$ be the expected squared error of the procedure fitted to m cases, for an independent new case $(\mathbf{X}_0, Y_0)$. Then

$$E(CV_K) = \text{Err}(n - n/K).$$

Proof. For $i \in I_k$, the rule $\hat{f}^{(-k)}$ is the procedure applied to $n - n/K$ independent cases, and case i is independent of them and has the same distribution as $(\mathbf{X}_0, Y_0)$. So each term of the double sum has expectation $\text{Err}(n - n/K)$. ■

The expectation is over training sets as well as new cases. Cross-validation therefore estimates the expected error of the *procedure* at a slightly smaller sample size, not the error of the particular fit computed from all n cases. For least squares with k normal regressors and a correct model, Theorem 14.2.6 gives $\text{Err}(m) = \sigma^2(1 + 1/m)(m - 2)/(m - k - 2)$, so the bias of CV_K can be computed exactly.

Example. Bias and spread of K-fold cross-validation

Take $n = 40$ cases with $k = 10$ independent standard normal regressors, an intercept, slopes

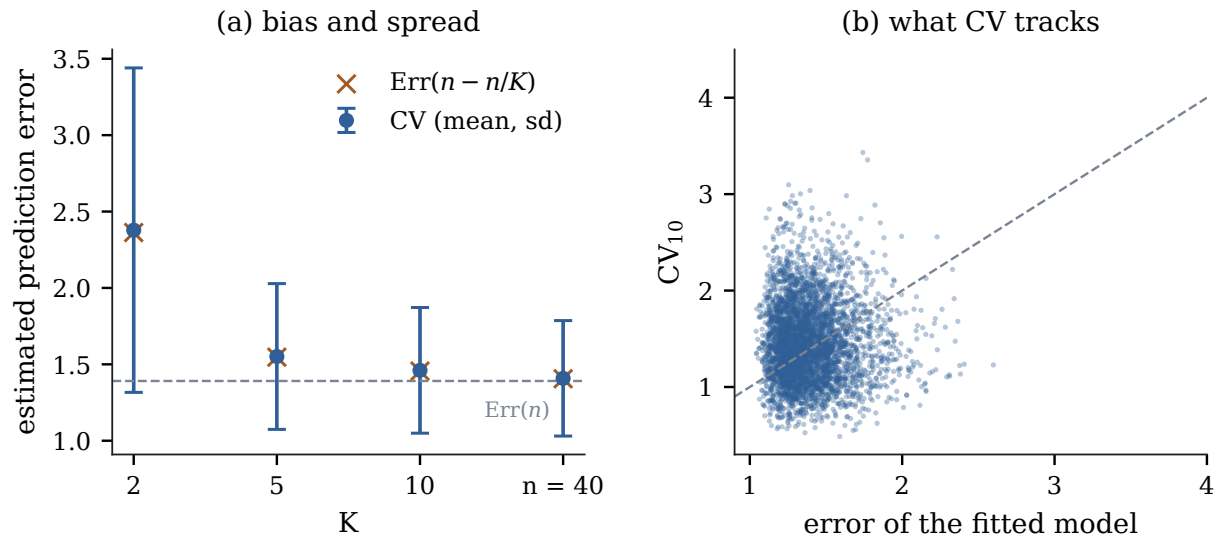


Figure 29.3.1: (a) Mean and standard deviation of CV_K over 4000 training sets ($n = 40$, ten regressors), with the expected error at sample size $n - n/K$ (crosses) and at n (dashed). (b) CV_{10} against the prediction error of the model actually fitted.

0.5 and $\sigma = 1$, and fit by least squares. The target is $\text{Err}(40) = 1.391$. Over 4000 training sets, the averages of CV_K for $K = 2, 5, 10, 40$ are 2.378, 1.551, 1.461 and 1.408, against $\text{Err}(n - n/K)$ of 2.363, 1.547, 1.456 and 1.406 from Proposition 29.3.1. Fitting to half the data inflates the error by 70% here, because ten slopes are estimated from twenty cases. Their standard deviations across training sets are 1.062, 0.477, 0.411 and 0.378. For this stable procedure leave-one-out has both the smallest bias and the smallest variance.

The error of the fitted model itself, $\sigma^2 + \|\hat{\beta} - \beta\|^2$ for a new standard normal case, varies too: its mean is 1.395 and its standard deviation 0.203. Panel (b) of Figure 29.3.1 plots CV_{10} against it. The correlation is -0.005 (-0.014 for leave-one-out). Cross-validation reports how well least squares does on average with forty such cases, not how well this fit will do.

```
import numpy as np
rng = np.random.default_rng(2905)
n_s, k_s, sigma = 40, 10, 1.0
beta = np.r_[1.0, np.full(k_s, 0.5)]           # intercept and ten
                                              slopes

def err_theory(m):
    """Expected error of a least squares fit to m cases (Chapter 14, normal
    regressors)."""
    return sigma**2 * (1 + 1 / m) * (m - 2) / (m - k_s - 2)

def cv_kfold(X, y, K):
    folds = np.arange(len(y)) % K               # equal folds (cases
    are exchangeable)
    sq = np.empty(len(y))
    for f in range(K):
        tr, te = folds != f, folds == f
        b, *_ = np.linalg.lstsq(X[tr], y[tr], rcond=None)
        sq[te] = (y[te] - X[te] @ b) ** 2
    return sq.mean()
```

```

Ks = [2, 5, 10, n_s]

def simulate(reps):
    cv = np.empty((reps, len(Ks)))
    err_cond = np.empty(reps)  # error of the fit at
    hand
    for r in range(reps):
        Xr = np.column_stack([np.ones(n_s), rng.normal(size=(n_s, k_s))])
        yr = Xr @ beta + sigma * rng.normal(size=n_s)
        b, *_ = np.linalg.lstsq(Xr, yr, rcond=None)
        err_cond[r] = sigma**2 + np.sum((b - beta) ** 2)  # new case  $X_0 \sim N(0, I)$ 
        cv[r] = [cv_kfold(Xr, yr, K) for K in Ks]
    return cv, err_cond

cv, err_cond = simulate(300)  # the book's figures
use 4000
for j, K in enumerate(Ks):
    print(f"K = {K:2d}: mean {cv[:, j].mean():.3f} (theory {err_theory(n_s - n_s //
    K):.3f}), sd {cv[:, j].std():.3f}")
print(f"error of the fit at hand: mean {err_cond.mean():.3f} (theory {err_theory(n_s
):.3f})")
print("correlation of CV_10 with it:", np.corrcoef(cv[:, 2], err_cond)[0, 1])

```

[Run this code online](#)

Bates, Hastie and Tibshirani (2024) study this phenomenon for linear models. The spread falling with K is typical of least squares, whose fits change little when a few cases are removed; for procedures that jump, such as subset selection, leave-one-out can be the more variable choice (Breiman and Spector 1992). Five or ten folds are the usual compromise.

29.3.2 The leverage shortcut

Leave-one-out looks expensive: n refits. For least squares and for ridge regression it costs one fit. The reason is a small fixed-point argument.

Theorem 29.3.2. Leave-one-out without refitting

Let Ω be a $p \times p$ nonnegative definite matrix, let $\hat{\beta}$ minimize $\|\mathbf{y} - \mathbf{X}\mathbf{b}\|^2 + \mathbf{b}^\top \Omega \mathbf{b}$, and let $\hat{\beta}_{(i)}$ minimize the same criterion with case i removed. Assume that $\mathbf{A}_{(i)} = \mathbf{X}_{(i)}^\top \mathbf{X}_{(i)} + \Omega$ is nonsingular for every i , where $\mathbf{X}_{(i)}$ is $\mathbf{X}$ without row i . Let $\mathbf{S} = \mathbf{X}(\mathbf{X}^\top \mathbf{X} + \Omega)^{-1} \mathbf{X}^\top$ and $\hat{\mathbf{y}} = \mathbf{S}\mathbf{y}$. Then:

- (a) $s_{ii} < 1$ for every i ;
- (b) $y_i - \mathbf{x}_{(i)}^\top \hat{\beta}_{(i)} = (y_i - \hat{y}_i)/(1 - s_{ii})$;
- (c) consequently

$$CV_n = \frac{1}{n} \sum_{i=1}^n \left(\frac{y_i - \hat{y}_i}{1 - s_{ii}} \right)^2. \quad (29.3.1)$$

For least squares ($\Omega = \mathbf{0}$) this is Theorem 20.3.1(d), and $CV_n = \text{PRESS}/n$ with PRESS as

in Eq. 20.3.1.

Proof.

- (a) Write $\mathbf{x} = \mathbf{x}_{(i)}$ and $\mathbf{A} = \mathbf{A}_{(i)}$, which is positive definite because it is nonnegative definite and nonsingular. Then $\mathbf{X}^\top \mathbf{X} + \mathbf{\Omega} = \mathbf{A} + \mathbf{x}\mathbf{x}^\top$, and $(\mathbf{A} + \mathbf{x}\mathbf{x}^\top)\mathbf{A}^{-1}\mathbf{x} = \mathbf{x}(1 + a)$ with $a = \mathbf{x}^\top \mathbf{A}^{-1}\mathbf{x} \geq 0$. Hence $(\mathbf{A} + \mathbf{x}\mathbf{x}^\top)^{-1}\mathbf{x} = \mathbf{A}^{-1}\mathbf{x}/(1 + a)$ and $s_{ii} = \mathbf{x}^\top (\mathbf{A} + \mathbf{x}\mathbf{x}^\top)^{-1}\mathbf{x} = a/(1 + a) < 1$.
- (b) Let $\tilde{y}_i = \mathbf{x}_{(i)}^\top \hat{\boldsymbol{\beta}}_{(i)}$, the leave-one-out prediction, and let $\tilde{\mathbf{y}}$ be $\mathbf{y}$ with y_i replaced by $\tilde{y}_i$. For any $\mathbf{b}$, the full criterion evaluated at $\tilde{\mathbf{y}}$ is the leave-one-out criterion plus $(\tilde{y}_i - \mathbf{x}_{(i)}^\top \mathbf{b})^2 \geq 0$. At $\mathbf{b} = \hat{\boldsymbol{\beta}}_{(i)}$ the extra term is zero and the leave-one-out criterion is at its minimum. So $\hat{\boldsymbol{\beta}}_{(i)}$ minimizes the full criterion for the data $\tilde{\mathbf{y}}$, and since that minimizer is unique ($\mathbf{X}^\top \mathbf{X} + \mathbf{\Omega} = \mathbf{A} + \mathbf{x}\mathbf{x}^\top$ is positive definite), $\mathbf{X}\hat{\boldsymbol{\beta}}_{(i)} = \mathbf{S}\tilde{\mathbf{y}}$. Its i th entry reads

$$\tilde{y}_i = (\mathbf{S}\tilde{\mathbf{y}})_i = (\mathbf{S}\mathbf{y})_i - s_{ii}(y_i - \tilde{y}_i) = \hat{y}_i - s_{ii}(y_i - \tilde{y}_i).$$

Rearranging, $(1 - s_{ii})(y_i - \tilde{y}_i) = y_i - \hat{y}_i$, and (a) allows division by $1 - s_{ii}$.

- (c) follows from (b), and for $\mathbf{\Omega} = \mathbf{0}$, $s_{ii} = h_{ii}$ and (b) is Theorem 20.3.1(d). ■

The argument uses only that deleting a case gives the same answer as replacing its response by its own prediction. It covers the ridge estimators of [Chapter 27](#) and the smoothers of [Chapter 43](#) (Theorem 43.5.2), but not the lasso or subset selection, where deleting a case can change which coefficients are nonzero.

[Section 27.2](#) already used the leave-one-out shortcut and GCV to choose the ridge λ ; the theorem proves the shortcut for every quadratic penalty. Replacing each s_{ii} by its average $\text{tr}(\mathbf{S})/n$ gives a criterion that needs only the trace.

Definition 29.3.2. Generalized cross-validation

For a linear smoother $\hat{\mathbf{y}} = \mathbf{S}\mathbf{y}$ with $\text{tr}(\mathbf{S}) < n$,

$$\text{GCV} = \frac{n^{-1}\|\mathbf{y} - \hat{\mathbf{y}}\|^2}{(1 - \text{tr}(\mathbf{S})/n)^2}.$$

Craven and Wahba (1979) introduced GCV for smoothing splines and Golub, Heath and Wahba (1979) for ridge regression, where it is leave-one-out after a rotation that equalizes the diagonal of $\mathbf{S}$. Since $(1 - x)^{-2} = 1 + 2x + O(x^2)$, $\text{GCV} \approx \overline{\text{err}}(1 + 2\text{df}/n)$, the covariance penalty Eq. 29.1.3 with σ^2 replaced by $\overline{\text{err}}$. For least squares $\text{GCV} = ns^2/(n - p)$, and for a correct model ($\boldsymbol{\mu} \in \mathcal{C}(\mathbf{X})$), by Proposition 20.3.2, $\text{E}(\text{PRESS})/n \geq \sigma^2 n/(n - p) = \text{E}(\text{GCV})$: high-leverage cases inflate leave-one-out.

For the state regression of Example (Violent crime in the states) with all five regressors, $\text{PRESS}/n = 0.1426$ and $\text{GCV} = 0.1176$. For ridge regression on the standardized regressors with $\lambda = 5$ (intercept unpenalized), $\text{tr}(\mathbf{S}) = 4.870$, and the shortcut gives $\text{CV}_n = 0.1309$, equal to the brute-force value from fifty refits to rounding error, against $\text{GCV} = 0.1140$.

```
import numpy as np
```

```

import statsmodels.api as sm
data = sm.datasets.statocrime.load_pandas().data.drop(index="District of Columbia")
names = ["hs_grad", "poverty", "single", "white", "urban"]
y = np.log(data["violent"].to_numpy())
Zs = data[names].to_numpy()
Zs = (Zs - Zs.mean(axis=0)) / Zs.std(axis=0)           # standardized regressors
X = np.column_stack([np.ones(len(y)), Zs])
n, p = X.shape
Omega = np.diag([0.0] + [1.0] * (p - 1))             # ridge penalty,
                                                    intercept left free

def ridge_fit(X, y, lam):
    return np.linalg.solve(X.T @ X + lam * Omega, X.T @ y)

lam = 5.0
S = X @ np.linalg.solve(X.T @ X + lam * Omega, X.T)   # the smoother matrix
resid = y - S @ y
loo_short = resid / (1 - np.diag(S))                  # the shortcut
loo_brute = np.array([y[i] - X[i] @ ridge_fit(np.delete(X, i, 0), np.delete(y, i),
    lam)
                    for i in range(n)])                # n refits
print("largest discrepancy:", np.max(np.abs(loo_short - loo_brute)))
cv_loo = np.mean(loo_short**2)
gcv = np.mean(resid**2) / (1 - np.trace(S) / n) ** 2
print(f"CV_n = {cv_loo:.5f}, GCV = {gcv:.5f}, tr(S) = {np.trace(S):.3f}")

```

[Run this code online](#)

Remark (Sketch: leave-one-out behaves like AIC). If every leverage is close to p/n , then $(1 - h_{ii})^{-2} \approx 1 + 2p/n$ and $CV_n \approx (SSE/n)(1 + 2p/n)$, so $n \log CV_n \approx n \log(SSE/n) + 2p$, the AIC of Eq. 29.2.2 up to a constant. Stone (1977) made this equivalence precise. Leave-one-out therefore shares AIC's large-sample behaviour: efficient when no candidate is correct (Li 1987), not consistent when one is. Shao (1993) showed that consistency requires leaving out a fraction of the cases that tends to one, the opposite of what prediction suggests; K -fold with fixed K is not consistent either.

29.3.3 Choosing a model by cross-validation

To choose among candidates, take the smallest CV_K . The fold errors $CV^{(k)}$ give a rough standard error $\text{sd}(CV^{(1)}, \dots, CV^{(K)})/\sqrt{K}$, which treats the folds as independent although their training sets overlap; no unbiased estimate of the variance of CV_K exists (Bengio and Grandvalet 2004). The **one-standard-error rule** (Breiman, Friedman, Olshen and Stone 1984) takes the simplest candidate within one standard error of the minimum, trading a little estimated error for a smaller model.

Example. Cross-validating the state regressions

For each size from 0 to 5, take the best subset of the state regressors by residual sum of squares and estimate its prediction error by ten-fold cross-validation, with one random split into folds of five states. Figure 29.3.2 shows the result. The minimum, 0.1136 with standard

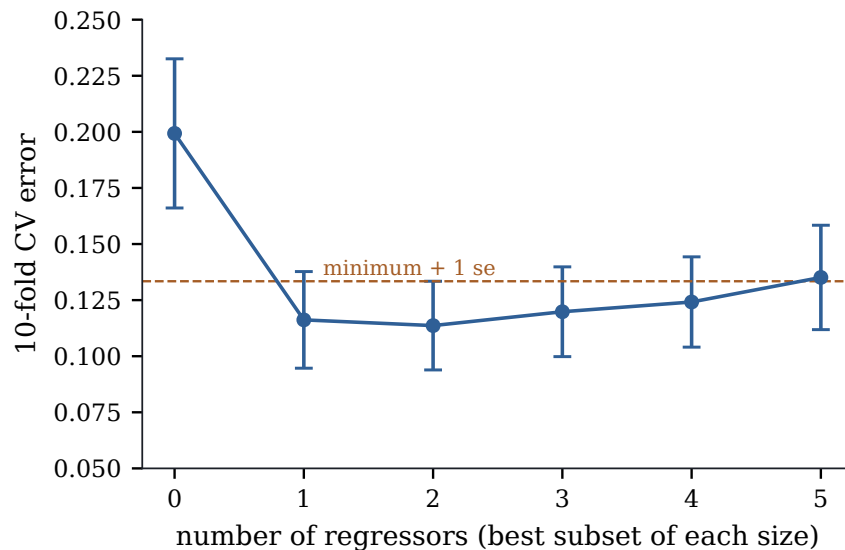


Figure 29.3.2: Ten-fold cross-validation error of the best subset of each size for log violent crime in the 50 states, with one-standard-error bars. Dashed: the minimum plus one standard error.

error 0.0198, is at two regressors (single parenthood and urbanization), the choice of C_p and AIC. Single parenthood alone has 0.1162, well within one standard error, so the one-standard-error rule chooses it, the choice of BIC. The full model has 0.1351 and the intercept alone 0.1993. Leave-one-out over all 32 candidates gives 0.1129 for single parenthood and urbanization and 0.1130 for single parenthood alone, a tie, and 0.1184 for poverty with single parenthood and urbanization.

```
import itertools
import numpy as np
import statsmodels.api as sm
data = sm.datasets.statecrime.load_pandas().data.drop(index="District of Columbia")
names = ["hs_grad", "poverty", "single", "white", "urban"]
y = np.log(data["violent"].to_numpy())
n = len(y)

def design(cols):
    return np.column_stack([np.ones(n)] + [data[c].to_numpy() for c in cols])

def sse(cols):
    X = design(cols)
    b, *_ = np.linalg.lstsq(X, y, rcond=None)
    return np.sum((y - X @ b) ** 2)

# best subset of each size, by residual sum of squares
best = [min(itertools.combinations(names, k), key=sse) for k in range(len(names) + 1)]

rng = np.random.default_rng(2906)
K = 10
folds = rng.permutation(np.arange(n) % K)  # a random split into 10
      folds of 5
cv, se = [], []
```

```

for cols in best:
    X = design(cols)
    fold_err = []
    for f in range(K):
        tr, te = folds != f, folds == f
        b, *_ = np.linalg.lstsq(X[tr], y[tr], rcond=None)
        fold_err.append(np.mean((y[te] - X[te] @ b) ** 2))
    cv.append(np.mean(fold_err))
    se.append(np.std(fold_err, ddof=1) / np.sqrt(K))          # the usual (optimistic)
                    standard error
cv, se = np.array(cv), np.array(se)
k_min = int(np.argmin(cv))
k_1se = int(np.min(np.nonzero(cv <= cv[k_min] + se[k_min])[0]))
for k, cols in enumerate(best):
    print(f"{k} regressors {cols}: CV {cv[k]:.4f} (se {se[k]:.4f})")
print("minimum CV at size", k_min, "; one-standard-error rule chooses size", k_1se)

```

[Run this code online](#)

The example has a flaw: the best subset of each size was found from all fifty states, and only its fitting was cross-validated.

29.3.4 Cross-validating the whole procedure

Cross-validation estimates the error of whatever is repeated inside the folds. Every decision that looked at the responses, such as screening regressors, choosing a subset, choosing λ or deleting outliers, must be repeated inside each fold. Otherwise the held-out cases have already influenced the fit that predicts them.

Example. Screening, then cross-validating

Take $n = 50$ cases, 200 candidate regressors and a response that is independent of all of them, with $\sigma = 1$. The procedure keeps the 5 regressors most correlated with the response and fits least squares on them. Every procedure has expected prediction error at least $\sigma^2 = 1$ here. Over 200 data sets:

- ten-fold cross-validation of the least squares fit, after screening on all the data, averages 0.720, and is below 1 in a fraction 0.93 of the data sets;
- ten-fold cross-validation that repeats the screening inside every fold averages 1.427;
- the actual prediction error of the screened and fitted model averages 1.372.

The wrong version reports that noise predicts the response. The right one is slightly pessimistic, because it uses 45 cases instead of 50 ([Proposition 29.3.1](#)).

```

import numpy as np
rng = np.random.default_rng(2907)
n, m, keep, K = 50, 200, 5, 10

def screen(X, y):
    """Indices of the `keep` columns most correlated with y."""
    Xc, yc = X - X.mean(axis=0), y - y.mean()
    corr = np.abs(Xc.T @ yc) / np.sqrt(np.sum(Xc**2, axis=0) * (yc @ yc))
    return np.argsort(-corr)[:keep]

```

```

def fit(X, y):
    A = np.column_stack([np.ones(len(y)), X])
    return np.linalg.lstsq(A, y, rcond=None)[0]

def predict(b, X):
    return b[0] + X @ b[1:]

def one_data_set():
    X = rng.normal(size=(n, m))
    y = rng.normal(size=n)                                # unrelated to every
    column
    folds = rng.permutation(np.arange(n) % K)
    cols = screen(X, y)                                    # screening on ALL the
    data
    wrong = right = 0.0
    for f in range(K):
        tr, te = folds != f, folds == f
        b = fit(X[tr][:, cols], y[tr])                    # wrong: screening done
        outside
        wrong += np.sum((y[te] - predict(b, X[te][:, cols])) ** 2)
        cols_f = screen(X[tr], y[tr])                     # right: screen again
        in the fold
        b = fit(X[tr][:, cols_f], y[tr])
        right += np.sum((y[te] - predict(b, X[te][:, cols_f])) ** 2)
    b = fit(X[:, cols], y)
    truth = 1 + b[0] ** 2 + np.sum(b[1:] ** 2)             # error on a new case
    return wrong / n, right / n, truth

res = np.array([one_data_set() for _ in range(200)])
print("average CV, screening outside the folds:", res[:, 0].mean().round(3))
print("average CV, screening inside the folds: ", res[:, 1].mean().round(3))
print("average true error of the fitted model: ", res[:, 2].mean().round(3))

```

[Run this code online](#)

When cross-validation is used both to choose among candidates and to report the error of the choice, the same problem appears in a milder form. The smallest of many cross-validation estimates is biased downwards as an estimate of the error of the model it picks (Section 29.5). The remedy is **nested cross-validation**: an outer loop holds out folds for assessment, and inside each outer training set an inner cross-validation makes the choice (Varma and Simon 2006). The outer estimate then describes the whole procedure, choice included, at sample size $n - n/K$.

29.3.5 Exercises

29.3.5.1 A. Check your understanding

Exercise. A1

In [Example \(Bias and spread of K-fold cross-validation\)](#), compute $\text{Err}(20)$ from Theorem 14.2.6 and confirm the value quoted for $K = 2$. Why is a holdout estimate that fits on half the

data especially biased when the number of regressors is large?

Solution. $\text{Err}(20) = (1 + 1/20)(18/8) = 1.05 \times 2.25 = 2.3625$. The factor $(m - 2)/(m - k - 2)$ blows up as the training size m approaches $k + 2$. With ten regressors, halving m from 40 to 20 moves it from 38/28 to 18/8.

29.3.5.2 B. Practice

Exercise. B1

For the intercept-only model, show that the leave-one-out residual is $n(y_i - \bar{y})/(n - 1)$ and that $\text{CV}_n = ns^2/(n - 1)$, where s^2 is the sample variance. Check this against [Theorem 29.3.2](#).

Solution. Without case i the mean is $\bar{y}_{(i)} = (n\bar{y} - y_i)/(n - 1)$, so $y_i - \bar{y}_{(i)} = (ny_i - n\bar{y})/(n - 1)$. Squaring and averaging, $\text{CV}_n = n^{-1}\{n/(n - 1)\}^2 \sum_i (y_i - \bar{y})^2 = n \sum_i (y_i - \bar{y})^2 / (n - 1)^2 = ns^2/(n - 1)$. In [Theorem 29.3.2](#), $s_{ii} = h_{ii} = 1/n$ and $1/(1 - 1/n) = n/(n - 1)$.

Exercise. B2

In [Theorem 29.3.2](#), suppose $\mathbf{X}$ has full column rank. Show that $s_{ii} \leq h_{ii}$, where h_{ii} is the least squares leverage, so that the shortcut inflates ridge residuals less than least squares residuals.

Solution. $\mathbf{X}^\top \mathbf{X} + \mathbf{\Omega} - \mathbf{X}^\top \mathbf{X} = \mathbf{\Omega}$ is nonnegative definite, so $(\mathbf{X}^\top \mathbf{X} + \mathbf{\Omega})^{-1} \leq (\mathbf{X}^\top \mathbf{X})^{-1}$ in the nonnegative definite ordering (for positive definite $\mathbf{A} \geq \mathbf{B}$, $\mathbf{B}^{-1} - \mathbf{A}^{-1} = \mathbf{B}^{-1/2}(\mathbf{I} - \mathbf{C}^{-1})\mathbf{B}^{-1/2}$ with $\mathbf{C} = \mathbf{B}^{-1/2}\mathbf{A}\mathbf{B}^{-1/2} \geq \mathbf{I}$). Evaluating both quadratic forms at $\mathbf{x}_{(i)}$ gives $s_{ii} \leq h_{ii}$.

29.3.5.3 C. Going deeper

Exercise. C1

Generalize [Proposition 29.3.1](#) to folds of unequal sizes $n_1, \dots, n_K$, and to leave-one-out for a design that is fixed rather than random. In the fixed-design case, what does $E(\text{CV}_n)$ equal for least squares, and how does it compare with the in-sample prediction error of [Theorem 29.1.1](#)?

Exercise. C2

Suppose $\max_i |h_{ii} - p/n| \leq \epsilon p/n$. Show that

$$\frac{\text{SSE}}{n} \left(1 - \frac{p}{n}(1 + \epsilon)\right)^{-2} \geq \text{CV}_n \geq \frac{\text{SSE}}{n} \left(1 - \frac{p}{n}(1 - \epsilon)\right)^{-2},$$

and deduce that $n \log CV_n = n \log(\text{SSE}/n) + 2p + O(p\epsilon + p^2/n)$. Which designs satisfy the hypothesis with small ϵ ?